GUIA DE ENDPOINTS
BACKEND – OFICINA MECÂNICA

Planejamento de API com Python + FastAPI
Rotas, responsabilidades, validações e divisão entre 6 integrantes

---

Projeto: Sistema Web para controle de veículos e ordens de serviço

Cenário: Cenário 2 – Oficina mecânica (conforme proposta da atividade)

Base: Proposta de Atividade N695 e protótipo “Oficina App — Esboço inicial”.

Documento de planejamento para distribuição das tarefas da equipe.

1. Objetivo e entendimento do trabalho

Este documento organiza as rotas do Backend da aplicação de oficina mecânica, relacionando cada endpoint às telas, botões e funcionalidades do protótipo. A proposta é permitir que os seis integrantes escolham responsabilidades claras e desenvolvam suas partes de forma coordenada.

O cenário escolhido é o Cenário 2 – Oficina mecânica. As duas entidades principais exigidas pela atividade são Veículo e Ordem de Serviço (OS): um veículo pode possuir várias ordens, e cada ordem pertence a um veículo.

A atividade exige CRUD completo (criar, listar, atualizar e excluir) para as duas entidades, validações específicas, persistência dos dados e integração da interface com o serviço Web. O protótipo também apresenta clientes, dashboard, itens/peças, agenda, login e pesquisa. Essas funcionalidades adicionais são separadas do escopo mínimo obrigatório.

Legenda de prioridade

Prioridade	Significado
Obrigatório	Necessário para atender ao escopo mínimo da atividade.
Complementar	Ajuda a reproduzir as funcionalidades visíveis no protótipo.
Opcional	Pode ser implementado se houver tempo e necessidade.

2. Convenções da API

Prefixo sugerido para todas as rotas: /api. Os métodos HTTP indicam o tipo de operação:

Método	Uso comum
GET	Consultar ou listar dados; não deve alterar o estado.
POST	Criar um registro ou executar uma ação.
PUT	Substituir/atualizar os dados de um registro.
PATCH	Atualizar parcialmente um registro, como seu status.
DELETE	Excluir um registro.

Os exemplos de JSON deste guia são propostas de contrato para a equipe alinhar. Os nomes e campos finais devem ser combinados antes do desenvolvimento para evitar incompatibilidade entre as partes.

3. Endpoints por funcionalidade

3.1 Veículos — obrigatório

Responsável pelo cadastro e gerenciamento dos veículos atendidos pela oficina.

Método	Endpoint	O que faz
GET	/api/veiculos	Lista os veículos cadastrados.
GET	/api/veiculos/{id}	Consulta os detalhes de um veículo.
POST	/api/veiculos	Cadastra um novo veículo.
PUT	/api/veiculos/{id}	Atualiza os dados de um veículo.
DELETE	/api/veiculos/{id}	Exclui um veículo, respeitando os relacionamentos.

Ligação com o Frontend: abrir a lista chama GET; salvar um novo cadastro chama POST; salvar alterações chama PUT; solicitar exclusão chama DELETE; abrir os detalhes chama GET por ID.

Exemplo de corpo para cadastrar veículo

```
{
```

```
  "placa": "QRS4F21",
  "marca": "Volkswagen",
  "modelo": "Gol 1.6",
  "ano": 2022,
  "quilometragem": 82300,
  "cliente_id": 1
}
```


O campo cliente_id é uma sugestão para relacionar o veículo ao proprietário, conforme a tela de clientes do protótipo.

3.2 Ordens de Serviço — obrigatório

Responsável pelo registro e acompanhamento dos serviços realizados nos veículos.

Método	Endpoint	O que faz
GET	/api/ordens-servico	Lista as ordens de serviço.
GET	/api/ordens-servico/{id}	Consulta uma OS específica.
POST	/api/ordens-servico	Cria uma nova OS.
PUT	/api/ordens-servico/{id}	Atualiza os dados de uma OS.
DELETE	/api/ordens-servico/{id}	Exclui uma OS.

Ligação com o Frontend: a tela de ordens chama GET; o botão “Nova ordem” chama POST; abrir a OS chama GET por ID; editar chama PUT; excluir chama DELETE.

Exemplo de corpo para criar uma OS


```
{
  "veiculo_id": 1,
  "quilometragem": 82300,
  "problema_relatado": "Barulho ao frear",
  "mecanico": "Rafael",
  "data_abertura": "2026-11-11T08:10:00",
  "data_previsao": "2026-11-11T17:00:00",
  "data_conclusao": null,
  "status": "EM_SERVICO",
  "observacoes": "Verificar pastilhas",
  "valor_total": 340.00
}
```


Serviços e peças podem ser armazenados em itens relacionados à OS. O valor total pode ser calculado pela soma dos itens, em vez de ser informado manualmente, desde que essa regra seja definida pela equipe.

3.3 Itens, serviços e peças da OS — complementar

O protótipo apresenta vários itens dentro de uma mesma ordem, como troca de óleo, óleo sintético e filtros, cada um com descrição, tipo e valor. Para representar isso, sugere-se uma entidade auxiliar ItemOrdemServico.

Método	Endpoint	O que faz
GET	/api/ordens-servico/{os_id}/itens	Lista os itens de uma OS.
POST	/api/ordens-servico/{os_id}/itens	Adiciona serviço ou peça à OS.
PUT	/api/ordens-servico/{os_id}/itens/{item_id}	Atualiza um item.
DELETE	/api/ordens-servico/{os_id}/itens/{item_id}	Remove um item.

Essa entidade auxiliar não substitui Veículo e Ordem de Serviço, que são as duas entidades principais exigidas na proposta da atividade.

3.4 Dashboard — complementar

O painel inicial exibe indicadores de ordens em serviço, aguardando peça, prontas, faturamento mensal e entregas previstas para hoje. Esses dados podem ser calculados a partir das ordens de serviço.

Método	Endpoint	O que faz
GET	/api/dashboard/resumo	Retorna indicadores gerais da oficina.

Método	Endpoint	O que faz
GET	/api/dashboard/entregas-hoje	Lista ordens com previsão de entrega hoje.
GET	/api/dashboard/ordens-recentes	Lista as ordens mais recentes.

Exemplo de resposta do resumo

```
{
  "em_servico": 12,
  "aguardando_peca": 4,
  "prontas": 3,
  "faturamento_mes": 18240.00
}
```

3.5 Ações e status das OS — complementar

Método	Endpoint	O que faz
PATCH	/api/ordens-servico/{id}/status	Altera parcialmente o status da OS.
POST	/api/ordens-servico/{id}/enviar-orcamento	Registra a ação de envio do orçamento.
POST	/api/ordens-servico/{id}/marcar-pronta	Marca a OS como pronta para entrega.

Ao clicar em “Marcar pronta”, por exemplo, o Frontend pode chamar POST /api/ordens-servico/1042/marcar-pronta. O Backend deve validar a transição permitida e atualizar o status para PRONTA. As rotas específicas são uma opção; a mesma operação também pode ser centralizada no PATCH de status.

3.6 Clientes — complementar

A tela de clientes mostra nome, telefone, e-mail, quantidade de veículos e histórico. Sugere-se uma entidade auxiliar Cliente para reproduzir essa parte do protótipo.

Método	Endpoint	O que faz
GET	/api/clientes	Lista clientes.
GET	/api/clientes/{id}	Consulta um cliente.
POST	/api/clientes	Cadastra um cliente.
PUT	/api/clientes/{id}	Atualiza os dados do cliente.
DELETE	/api/clientes/{id}	Exclui cliente, respeitando vínculos.
GET	/api/clientes/{id}/veiculos	Lista veículos de um cliente.
GET	/api/clientes/{id}/historico	Consulta o histórico de OS do cliente.

3.7 Pesquisa e filtros — complementar

Os filtros podem ser implementados com query parameters nas rotas existentes. Não é necessário criar uma rota diferente para cada filtro.

Requisição	Resultado
GET /api/ordens-servico?status=EM_SERVICO	Filtra ordens por status.
GET /api/ordens-servico?placa=QRS4F21	Filtra ordens pela placa.
GET /api/ordens-servico?cliente=Marina	Filtra ordens pelo cliente.
GET /api/veiculos?placa=QRS4F21	Busca veículos pela placa.
GET /api/clientes?nome=Marina	Busca clientes pelo nome.
GET /api/busca?q=QRS4F21	Pesquisa geral por placa, cliente ou número da OS.

As cinco primeiras linhas são variações de filtros em rotas já listadas. A rota /api/busca é uma rota adicional de pesquisa geral.

3.8 Autenticação — opcional

Método	Endpoint	O que faz
POST	/api/auth/login	Valida credenciais e autentica o usuário.
POST	/api/auth/esqueci-senha	Solicita recuperação de senha.
GET	/api/auth/me	Retorna os dados do usuário autenticado.

3.9 Agenda — complementar

Método	Endpoint	O que faz
GET	/api/agenda	Consulta ordens organizadas por data.

Exemplo: GET /api/agenda?data=2026-11-11 retorna ordens previstas para a data informada.

3.10 Exportação — opcional

Método	Endpoint	O que faz
GET	/api/ordens-servico/exportar	Exporta a listagem de ordens, por exemplo em CSV.

4. Resumo consolidado

Grupo	Quantidade de rotas
Veículos	5
Ordens de serviço	5
Itens da OS	4
Dashboard	3
Ações da OS	3
Clientes	7
Pesquisa geral	1 rota própria; filtros reutilizam rotas existentes
Autenticação	3
Agenda	1
Exportação	1
Total	33 rotas distintas, contando a pesquisa geral e sem contar filtros como novas rotas.

Os 10 endpoints fundamentais são os cinco de Veículos e os cinco de Ordens de Serviço. As demais rotas ampliam o protótipo e devem ser priorizadas conforme o tempo disponível.

5. Divisão entre os seis integrantes

A divisão abaixo é uma proposta de organização. O integrante 6 tem uma função transversal de banco de dados, integração e testes; é importante que ele comece desde o início, e não apenas quando os demais terminarem.

Integrante 1 — Veículos

CRUD de veículos e validação da placa e dos campos obrigatórios.

Rotas/entregas: GET /api/veiculos; GET /api/veiculos/{id}; POST /api/veiculos; PUT /api/veiculos/{id}; DELETE /api/veiculos/{id}

Integrante 2 — Ordens de Serviço

CRUD de OS e associação da ordem ao veículo.

Rotas/entregas: GET /api/ordens-servico; GET /api/ordens-servico/{id}; POST /api/ordens-servico; PUT /api/ordens-servico/{id}; DELETE /api/ordens-servico/{id}

Integrante 3 — Itens e ações da OS

Itens/peças/serviços, status e regras de negócio da OS.

Rotas/entregas: GET /api/ordens-servico/{os_id}/itens; POST /api/ordens-servico/{os_id}/itens; PUT /api/ordens-servico/{os_id}/itens/{item_id}; DELETE /api/ordens-servico/{os_id}/itens/{item_id}; PATCH /api/ordens-servico/{id}/status; POST /api/ordens-servico/{id}/enviar-orcamento; POST /api/ordens-servico/{id}/marcar-pronta

Integrante 4 — Dashboard, pesquisa e agenda

Consultas para o painel, pesquisa geral, filtros e agenda.

Rotas/entregas: GET /api/dashboard/resumo; GET /api/dashboard/entregas-hoje; GET /api/dashboard/ordens-recentes; GET /api/busca; GET /api/agenda

Integrante 5 — Clientes e autenticação

CRUD de clientes, consultas relacionadas e autenticação, se implementada.

Rotas/entregas: GET /api/clientes; GET /api/clientes/{id}; POST /api/clientes; PUT /api/clientes/{id}; DELETE /api/clientes/{id}; GET /api/clientes/{id}/veiculos; GET /api/clientes/{id}/historico; POST /api/auth/login; POST /api/auth/esqueci-senha; GET /api/auth/me

Integrante 6 — Banco, integração e testes

Modelagem do banco, relacionamentos, camada de persistência, configuração do FastAPI, Swagger, testes e integração. Também pode implementar a exportação CSV.

Rotas/entregas: GET /api/ordens-servico/exportar

6. Validações e tratamento de erros

A proposta da atividade define quatro regras específicas para o cenário de oficina mecânica. Elas devem ser validadas no Backend, mesmo que o Frontend também valide os campos.

Regra	Onde aplicar
Campos obrigatórios	Cadastro e atualização de veículos e ordens de serviço.
Placa em formato válido	Cadastro e atualização de veículos.
Valor do serviço maior que zero	Cadastro e atualização dos itens/serviços da OS.
Data de conclusão não anterior à data de abertura	Cadastro e atualização da OS.

Recomendações adicionais de tratamento de erros

Código HTTP	Quando utilizar
200 OK	Consulta ou atualização realizada com sucesso.
201 Created	Registro criado com sucesso.
204 No Content	Exclusão realizada sem corpo de resposta, se esse padrão for adotado.
404 Not Found	Veículo, cliente, OS ou item não encontrado.
409 Conflict	Conflito de relacionamento ou tentativa de exclusão incompatível com os vínculos.
422 Unprocessable Entity	Dados de entrada inválidos; o FastAPI também usa esse status em validações padrão.

Recomendação: impedir a exclusão de um veículo que possua ordens de serviço, ou definir uma estratégia que preserve o histórico. Para clientes, também deve ser respeitada a existência de veículos vinculados.

7. Estrutura sugerida do projeto FastAPI

Uma estrutura em camadas ajuda os integrantes a trabalharem em paralelo e atende às práticas de Clean Code destacadas na atividade.

```
backend/
├── main.py
├── database/
│   ├── connection.py
│   └── models.py
├── schemas/
│   ├── veiculo_schema.py
│   ├── ordem_servico_schema.py
│   ├── cliente_schema.py
│   └── item_os_schema.py
├── routers/
│   ├── veiculos.py
│   ├── ordens_servico.py
│   ├── clientes.py
│   ├── dashboard.py
│   ├── autenticacao.py
│   ├── agenda.py
│   └── busca.py
├── services/
│   ├── veiculo_service.py
│   ├── ordem_servico_service.py
│   ├── cliente_service.py
│   └── dashboard_service.py
├── repositories/
│   ├── veiculo_repository.py
│   ├── ordem_servico_repository.py
│   └── cliente_repository.py
└── tests/
    ├── test_veiculos.py
    ├── test_ordens_servico.py
    └── test_clientes.py
```

Responsabilidade de cada camada

Camada	Responsabilidade
Routers	Recebem requisições HTTP e definem os endpoints.
Services	Executam as regras de negócio e coordenam operações.
Repositories	Acessam e manipulam os dados persistidos.
Schemas	Definem e validam os formatos de entrada e saída.
Database/Models	Configuram conexão e representam as tabelas/entidades.

8. Ordem recomendada de desenvolvimento

Prioridade	O que desenvolver
1 — Obrigatório	CRUD completo de Veículos e Ordens de Serviço; relacionamento entre elas; validações exigidas; persistência.
2 — Reproduzir protótipo	Dashboard, clientes, itens/serviços da OS, alteração de status, agenda e pesquisa.
3 — Adicionais	Login/recuperação de senha, exportação e estoque.

A tela de estoque aparece no protótipo, mas o esboço não apresenta detalhes suficientes para definir seu CRUD. Por isso, não foram propostos endpoints de estoque neste guia.

9. Acordos que a equipe deve fechar antes de programar

1. Definir os nomes finais dos campos e os tipos de dados dos modelos.
2. Definir se Cliente será uma entidade auxiliar e como será associado ao Veículo.
3. Definir os estados possíveis da OS (por exemplo: ORCAMENTO, EM_SERVICO, AGUARDANDO_PECA,

PRONTA e ENTREGUE).
4. Definir como o valor total será calculado a partir dos itens.
5. Definir o banco de dados e a estratégia de migração/criação das tabelas.
6. Definir exemplos de requisição e resposta para cada rota compartilhada.
7. Combinar padrões de nomes, branches, commits e revisão de código.

10. Observações finais

A lista de 33 rotas é um planejamento ampliado para aproximar o Backend do protótipo. Ela não significa que todas sejam obrigatórias para a avaliação. O núcleo exigido pela proposta é o CRUD das duas entidades relacionadas — Veículo e Ordem de Serviço — com validações e persistência.

A divisão entre seis integrantes é uma sugestão. Como o trabalho depende de integração, todos devem compartilhar desde o início os schemas, os nomes dos campos, os relacionamentos e os formatos de resposta. O integrante responsável por banco e integração deve trabalhar em paralelo com os demais.

Fontes utilizadas: “N695 – Proposta de Trabalho 2026_2” (especialmente seções 3, 4, 5 e 8) e “Oficina App — Esboço inicial” (telas e funcionalidades do protótipo). Campos de JSON, rotas complementares, códigos HTTP e arquitetura de pastas são propostas técnicas para organização da equipe, não especificações explícitas dos PDFs.